Project Report: PIC16F877A Four-Function Calculator Simulation

1. Project Overview & Objectives

This project implements a complete 4-function digital calculator (+, -, *, /) utilizing a PIC16F877A microcontroller and an LCD display simulated within Proteus. The system allows users to input multi-digit operands via a physical push-button matrix layout, select arithmetic operations, and read real-time computed outputs directly on the screen.

2. Hardware Architecture & Working Principle

Core Components

• Microcontroller: PIC16F877A (Central processing unit managing input decoding, arithmetic logic, and display rendering).

• Display: 16x2 LCD screen (interfaced via 4-bit/8-bit mode on PORTD for visual feedback).

How the Input Mechanism Works

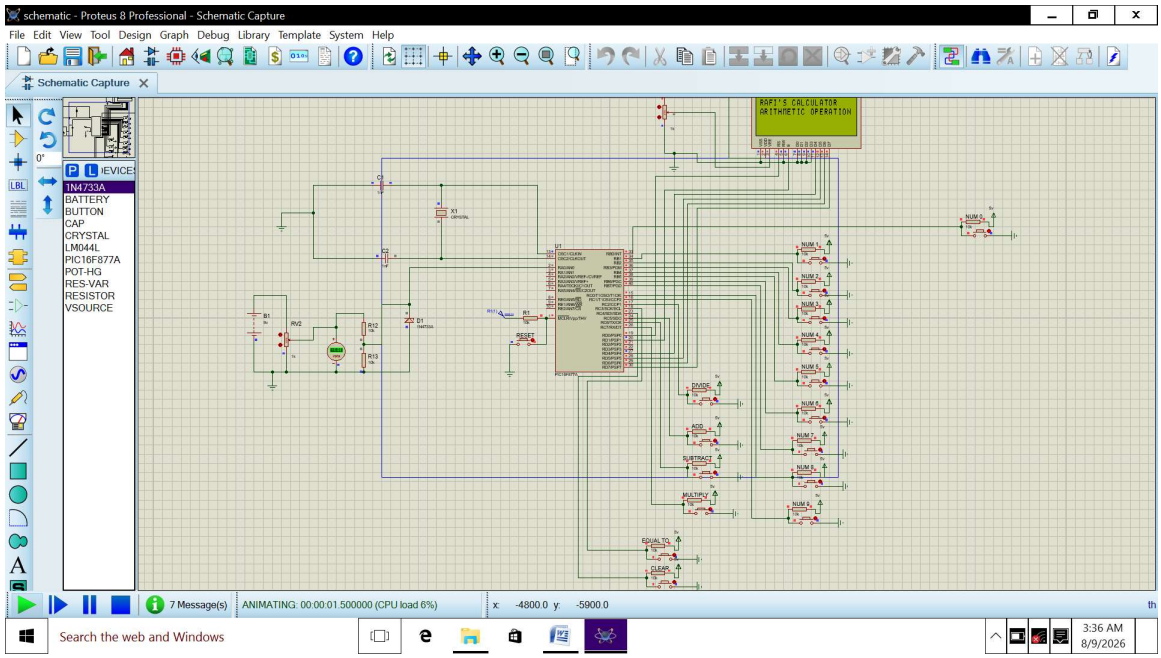
The input interface relies on active-high push-button architecture paired with 10k pull-down resistors tied to ground:

• In their default, untouched state, the microcontroller pins read a stable Logic LOW (0).

• When a user presses a push button, it connects the respective microcontroller pin to a 5V source, forcing an input signal state change to Logic HIGH (1).

• The software constantly polls these ports (read_key function) to detect the active transition, implements a software debounce delay to filter mechanical noise, and registers the respective numeric or operational command.

3. Proteus Simulation Setup



4. Software Implementation & Language

Programming Language

The embedded software was written in standard C language and compiled using the MikroC PRO for PIC toolchain.

Overcoming the MikroC Demo Limit

During the initial development phase, the project encountered constraints associated with the MikroC compiler's trial/demo version (which imposes a strict 2KB output code-size limit). To successfully bypass and manage this limitation for GitHub deployment without hitting compilation blocks, the following strategies were applied:

• Optimized Code Structure: Avoided heavy, redundant standard library calls where lightweight custom functions (such as custom light_LongToStr routine) could perform the task natively.

• Streamlined Logic: Kept the state machine concise, removing extraneous abstractions and keeping the binary footprint well under the threshold.

5. Essential Microcontroller Code Breakdown (For GitHub)

Key Polling and Operator Allocation (read_key)

This function scans the designated pins across PORTB and PORTC (including your added operator pins on RC5, RC6, and RC7) to return a unique identifier code for each button:

```
// Operators and Commands mapped to PORTC
if (PORTC.F2 == 0) { return 10; } // Division (/)
if (PORTC.F3 == 0) { return 11; } // Equals (=)
if (PORTC.F4 == 0) { return 12; } // Clear (C)
if (PORTC.F5 == 0) { return 13; } // Addition (+)
if (PORTC.F6 == 0) { return 14; } // Subtraction (-)
if (PORTC.F7 == 0) { return 15; } // Multiplication (*)
```

State Machine & Arithmetic Execution

The main loop processes input sequentially:

1. Stage 0: Captures digits for num1 until an operator (+, -, *, or /) is selected.

2. Stage 1: Switches tracking state to capture num2.

3. Evaluation: Once = is pressed, the system evaluates the current_op variable, executes the arithmetic operation using signed long variables (allowing negative results for subtractions like 5 - 10 = -5), and updates the LCD screen formatting.